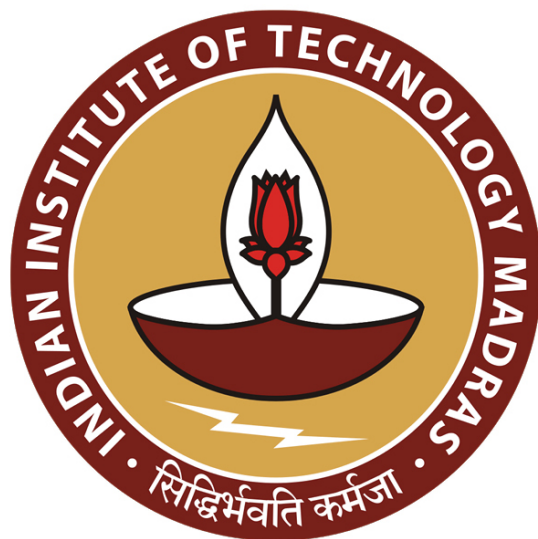


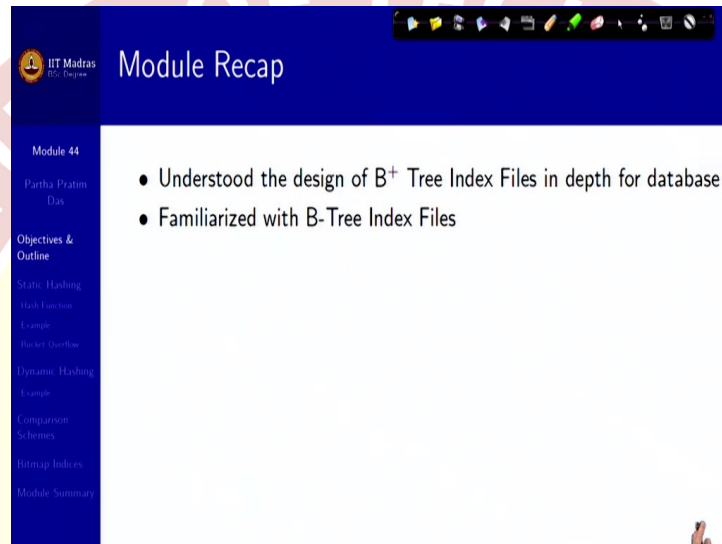


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Indexing and Hashing/4: Hashing

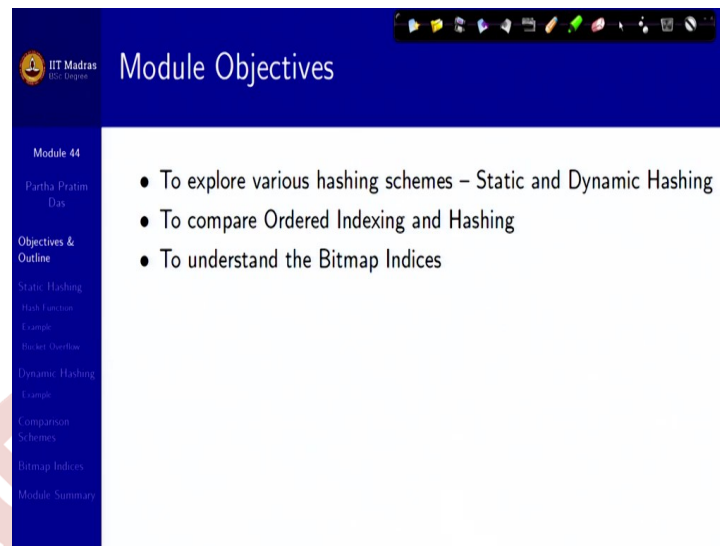
Welcome to Module 44 of Database Management Systems course in the IIT Madras online B.Sc. program.

(Refer Slide Time: 0:28)



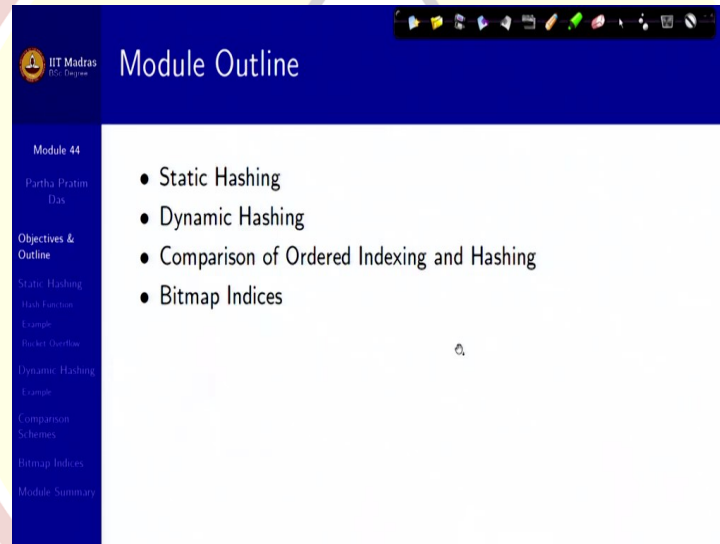
We have been discussing about indexing and hashing and we did the first three modules we have done in this week on B+ tree, I mean culminating in B+ tree index files for persistent data store. And we looked at B-tree also in the last module, but our primary objective was to create index files and data files with efficient structure which is B+ tree as we have learned.

(Refer Slide Time: 0:59)



Module Objectives

- To explore various hashing schemes – Static and Dynamic Hashing
- To compare Ordered Indexing and Hashing
- To understand the Bitmap Indices



Module Outline

- Static Hashing
- Dynamic Hashing
- Comparison of Ordered Indexing and Hashing
- Bitmap Indices

We will introduce another concept here which can help in indexing operations known as hashing, you may be familiar with little bit of hashing, hashtag, hash table, hash maps and so on. So, we will talk about static and dynamic hashing and compare between ordered indexing that we have discussed so far and hashing and introduce the concept of bitmap indices as well.

(Refer Slide Time: 1:28)

Static Hashing

Module 44
Partha Pratim Das
Objectives & Outline
Static Hashing
Hash Function
Example
Bucket Overflow
Dynamic Hashing
Example
Comparison Schemes
Bitmap Indices
Module Summary

Hash Function

- A hash function h maps data of arbitrary size (from domain D) to fixed integers from 0 to $N - 1$

$$h : D \rightarrow [0..N]$$

- Given key k , $h(k)$ is called hash values, hash codes, digests, or simply
- If for two keys $k_1 \neq k_2$, we have $h(k_1) = h(k_2)$, we say a collision has
- A hash function should be *Collision Free* and *Fast*

Diagram illustrating a hash function mapping keys to hashes:

keys	hash function	hashes
John Smith	hash function	00
Lisa Smith		01
		02
		03

Handwritten notes: 2^0 and 2^0 are written next to the hash values 00 and 01 respectively.

So, static hashing, the hashing per say, so what is hashing? In hashing we have a domain of values, which is normally very large. So, if that domain were small, then we could take each and every value and used it as an array index to do a random access. It is again, how do you make search efficient? That is the basic question. We could keep the data in an array, so that the value of that index coming from the set of possible values in D could be used at the address.

But usually, that is not the case. Often times, the, this domain of possible values is too large. And you have very small actual data that you use. So, think about names, they could be of practically any length, even if you put some limits, so it can go up to 30 to 40 characters very easily for many people.

So, you can think of if I say it even if it is 20 characters, and you have 26 letters in the alphabet and the blank, so 27. So, it is the first place can be taken in 27 ways, second in another 27 ways and so on. So, it is 27 to the power 20 is the kind of size of the space that you are talking of, but you will possibly have only a few 1000 of them.

(Refer Slide Time: 3:26)

IIT Madras
BSc Degree

Hash Function

- A hash function h maps data of arbitrary size (from domain D) to fixed integers from 0 to $N > 0$

$$h : D \rightarrow [0..N]$$

- Given key k , $h(k)$ is called hash values, hash codes, digests, or simply
- If for two keys $k_1 \neq k_2$, we have $h(k_1) = h(k_2)$, we say a collision has
- A hash function should be *Collision Free* and *Fast*

IIT Madras
BSc Degree

Hash Function

- A hash function h maps data of arbitrary size (from domain D) to fixed integers from 0 to $N > 0$

$$h : D \rightarrow [0..N]$$

- Given key k , $h(k)$ is called hash values, hash codes, digests, or simply
- If for two keys $k_1 \neq k_2$, we have $h(k_1) = h(k_2)$, we say a collision has
- A hash function should be *Collision Free* and *Fast*

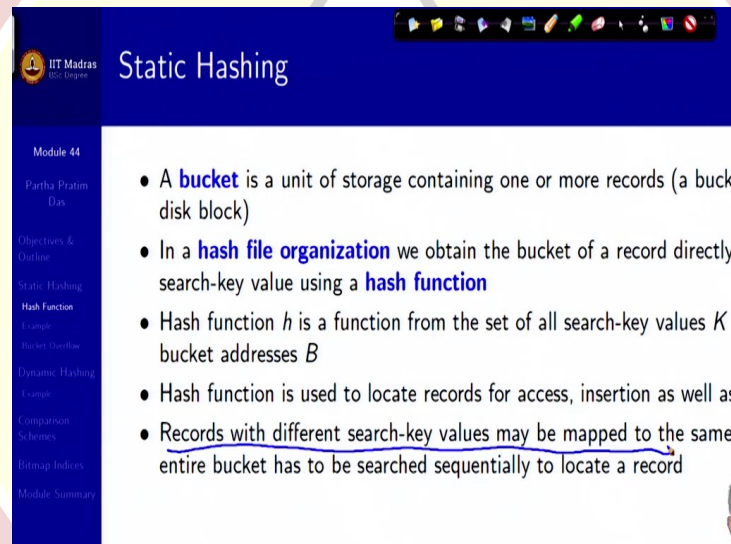
So, you take, instead of using D as an index, I mean values of D as an index, you use a function that maps the D on to a range of small range of values. So, you have different names here, you map them to a range of say 16 values called hash values hash keys, so that this function does something with the value in from the D and produces a value in between 0 to n .

So, this is an example being shown. So, John has a, gets a value 2 so that is where you are expected to get the details about John. Lisa has a value 1, this is where you get the details,

Sam has a value 4, so this is where you get the details. Sandra incidentally, under this function has a value 2. So, two distinct keys John Smith and Sandra Dee both actually generate the same hash value.

So, if I have two keys, which are unequal and if they have the same hash value, we say a collision has happened. A collision has taken place. If a collision has taken place, then we have not been able to resolve and find, we will still have to go something else to do that. But this is, this will have to be a limitation or this will have to be a basic property of the hash function because you are taking a large domain of values in D , and mapping it to a small domain of $n+1$ value. So, there will be some for some keys, it will always have a collision and how do we manage that efficiently is the, will give the efficacy of the hashing.

(Refer Slide Time: 5:21)



The screenshot shows a presentation slide titled "Static Hashing" from IIT Madras. The slide is part of Module 44, presented by Partha Pratim Das. The left sidebar lists the following topics: Objectives & Outline, Static Hashing, Hash Function, Examples, Bucket Overflow, Dynamic Hashing, Examples, Comparison Schemes, Bitmap Indexes, and Module Summary. The main content area contains the following bullet points:

- A **bucket** is a unit of storage containing one or more records (a bucket disk block)
- In a **hash file organization** we obtain the bucket of a record directly search-key value using a **hash function**
- Hash function h is a function from the set of all search-key values K bucket addresses B
- Hash function is used to locate records for access, insertion as well as
- Records with different search-key values may be mapped to the same entire bucket has to be searched sequentially to locate a record

So, the values, the index that a hash actually takes you to, maps you to is a bucket, it is considered a bucket where 1 or more records can be stored. Typically, we take the bucket to be a disk block again, the question of transferring between the memory and so on. So, a hash file organization will have this hashing function. So, you have the key, you do the hashing, you get the bucket and there you find the record.

Now, naturally, as we have seen records with different key values may be mapped to the same bucket. So, the bucket has to be searched sequentially to locate a record. So, in one part you gain by quickly coming to a subset of values; in the other part you will have to pay if your bucket is very full, or if you have too many buckets at a given hash value.

(Refer Slide Time: 6:25)

Example of Hash File Organization

Hash file organization of *instructor* file, using dept.name as key

- There are 10 buckets
- The binary representation of the i^{th} character is assumed to be the in
- The hash function returns the sum of the binary representations of th
modulo 10
 - For example

$$\begin{aligned} h(\text{Music}) &= 1 & h(\text{History}) &= 2 \\ h(\text{Physics}) &= 3 & h(\text{Elec. Eng.}) &= 3 \end{aligned}$$

Music (handwritten note with two arrows pointing to the first example)

Example of Hash File Organization

Hash file organization of *instructor* file, using dept.name as key

- There are 10 buckets
- The binary representation of the i^{th} character is assumed to be the in
- The hash function returns the sum of the binary representations of th
modulo 10
 - For example

$$\begin{aligned} h(\text{Music}) &= 1 & h(\text{History}) &= 2 \\ h(\text{Physics}) &= 3 & h(\text{Elec. Eng.}) &= 3 \end{aligned}$$

So, example, so our instructor table using the department name as a key. So, as a department name is a key so, let us say there are 10 buckets. So, what we do is the department name, say if I say Music, then each one of them has a ASCII value; binary representation, we take that as an integer. And then we add them together, sum them together the ASCII of m plus ASCII a u plus ASCII of s like that, it becomes a large number.

And then we do a modulo that is we divide by 10 and take the remainder because there are 10 buckets, so that 10 buckets the indices are 0 to 9. So, if I do a modulo, I will get a value between 0 to 9 and I use that as a value. You can check out but these are the computations of different names of departments that we have hashed through this process that you take the

ASCII value of every character in the name as an integer, add them, divide by 10 and take that residue, take the remainder.

(Refer Slide Time: 7:54)

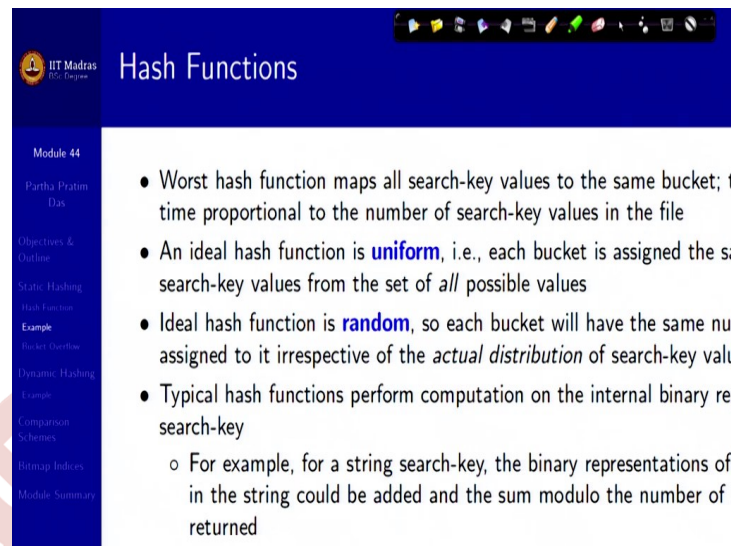
Example of Hash File Organization

bucket 0	bucket 1	bucket 2	bucket 3	bucket 4	bucket 5	bucket 6	bucket 7
	15151 Mozart Music 40000	32343 El Said History 80000	22222 Einstein Physics 95000	12121 Wu Finance 90000	76766 Crick Biology 72000	10101 Srinivasan Comp. Sci. 65000	
		58583 Califiori History 60000		76543 Singh Finance 80000		45565 Katz Comp. Sci. 75000	
						83821 Brandt Comp. Sci. 92000	

So, we will keep the records according to this. So, there are different buckets 8 are shown here right now. So, Music has a hash value of 1. So, it goes here. History has hash value of 2, it goes here. Computer science has hash value of 6, it goes here. Interestingly physics and electrical engineering both have hash value 3. So, both of them go here. So, once you come here, you will again have to search linearly you do not know.

So, this is the collision that is happening, this is what puts two or more distinct values in the same bucket, at this is how it can be organized and it can see that while searching linearly within a bucket is certainly over it, but it is also true that if I have an appropriate hash function, then out of n bucket possibilities n-1 have got eliminated. So, there is a large number of keys which have got eliminate. So, the linear search even though it is required will be done on a very small set, relatively small set.

(Refer Slide Time: 9:06)



IIT Madras

Hash Functions

Module 44

Parth Pratim Das

Objectives & Outline

Static Hashing

Hash Function

Example

Binary Hashing

Dynamic Hashing

Example

Comparison Schemes

Bitmap Indexes

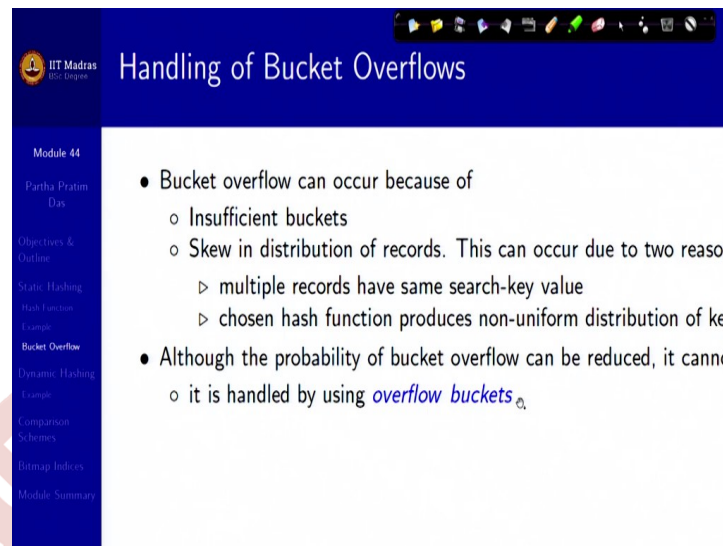
Module Summary

- Worst hash function maps all search-key values to the same bucket; time proportional to the number of search-key values in the file
- An ideal hash function is **uniform**, i.e., each bucket is assigned the same number of search-key values from the set of *all* possible values
- Ideal hash function is **random**, so each bucket will have the same number of search-key values assigned to it irrespective of the *actual distribution* of search-key values
- Typical hash functions perform computation on the internal binary representation of the search-key
 - For example, for a string search-key, the binary representations of the characters in the string could be added and the sum modulo the number of buckets is returned

Now, naturally there are other than collision, there are certain naturally we would want hash functions, which do not give a lot of collision. So, what will be, there will be some other desirable properties for example, in the presence of collision, which will, which is unavoidable the number of keys that map to a bucket should be more or less uniform. If it is not, then one bucket will start growing larger and larger and it will overflow the block size.

So, I have to have another block as an extension of that bucket and so on so forth. So, it this uniformity of hash values is a very desirable property of a hash function. Similarly, a hash function is called ideal if the, every bucket will have the same number of keys and typically, only random functions are behave in that way and of course, we cannot just work with random functions as such.

(Refer Slide Time: 10:27)



IIT Madras

Handling of Bucket Overflows

Module 44

Partha Pratim Das

Objectives & Outline

Static Hashing

Hash Function

Example

Bucket Overflow

Dynamic Hashing

Example

Comparison Schemes

Bitmap Indexes

Module Summary

- Bucket overflow can occur because of
 - Insufficient buckets
 - Skew in distribution of records. This can occur due to two reasons
 - ▷ multiple records have same search-key value
 - ▷ chosen hash function produces non-uniform distribution of key
- Although the probability of bucket overflow can be reduced, it cannot
 - it is handled by using *overflow buckets*.

Now, if a bucket has too many entries, it could happen in multiple ways, one could be because you have insufficient number of buckets that is you are trying to map to a very small range. So, very soon you will have much higher amount of collision and buckets will become full very easily. It could happen because the distribution is skewed it is not uniform or not nowhere near uniform.

There are multiple records which have the same key value. For example, let us say we were looking at the department name and let us say the mechanical engineering department have 150 instructors whereas Music Department has 5 instructors, naturally the data inherently is skewed.

So, either you have, you know the statistics of this data distribution and use that in the hash function or you will get naturally very skewed result in terms of number of records going to the same bucket. It could also happen because of the choice of the hash function, because some hash functions give a better uniform distribution some give less, but it will, it is a situation we will have to live with all the time.

(Refer Slide Time: 11:48)

Handling of Bucket Overflows (2)

- **Overflow chaining** – the overflow buckets of a given bucket are chained in a linked list
- Above scheme is called **closed hashing**
 - An alternative, called **open hashing**, which does not use overflow buckets is suitable for database applications

Diagram illustrating overflow chaining:

```
graph LR; B0[bucket 0] --> B1[bucket 1]; B1 --> O1[overflow buckets for bucket 1]; O1 --> B2[bucket 2];
```

The diagram shows three main buckets labeled bucket 0, bucket 1, and bucket 2. An arrow points from bucket 1 to a group of two boxes labeled 'overflow buckets for bucket 1'. Another arrow points from bucket 2 to the first overflow bucket of bucket 1, indicating a linked list structure for overflow handling.

So, the choice is to have overflow buckets that you need false here you are filled very naturally, you will have a pointer to go to the next of course, this is increasing your cost dramatically now, because you are searching through a bucket in a linear form. And as you do that, so, you have more and more in the chain, you have those many additional block transfers to be done to memory because if every block, every bucket is the size of a block, one you have brought you did not find it there, you have to go to the next again keep on doing this.

So, overflow handling is a problem in this scheme, which is otherwise called a closed hashing scheme. There are alternates to that, but they are not suitable for database application.

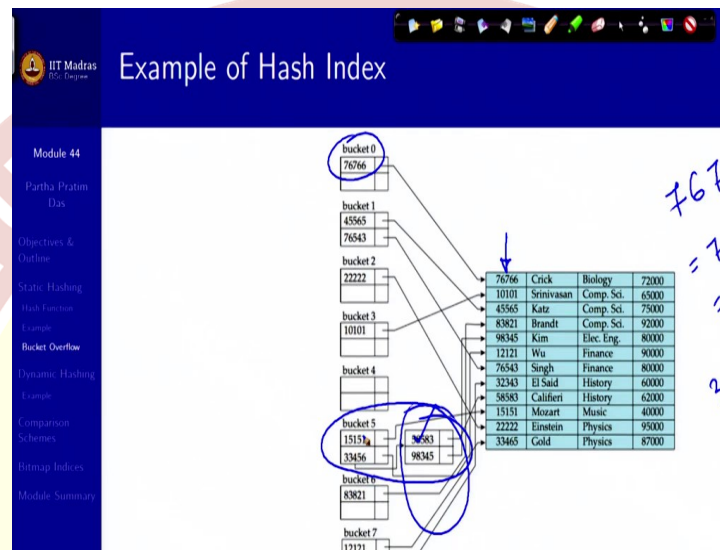
(Refer Slide Time: 12:43)

Hash Indices

- Hashing can be used not only for file organization, but also for indexing
- A **hash index** organizes the search keys, with their associated record hash file structure
- Strictly speaking, hash indices are always secondary indices
 - if the file itself is organized using hashing, a separate primary hash the same search-key is unnecessary
 - However, we use the term hash index to refer to both secondary and hash organized files

So, this hash indices. This hash values I mean just to note that can be used not only for file organization, but you can use it for basic index structure organization also, because that is quite understandable, because in either case, you are basically doing a search operation. And hashing is a shortcut, efficient shortcut for searching a key from a appropriate bucket with the power of this mathematical hash function.

(Refer Slide Time: 13:26)



So, this is an example here of the earlier one here, we have organized the records according to the ID of the employee going to different buckets, and the computation is you add the digits. So, for 76766, you add the digit, $7+6$; $13+7$; $20+6$; $26+6$; 32. So, 30, you get 32. And then you do 32. Here, you can see mod 8, so which will give you 0, so bucket 0 has this. So, that is how you do.

Now, you would normally expect this to be pretty uniform across values, but the fact is the, this kind of IDs are not generated in that way, it may have some other bias to do that. So, given this data, you can see that bucket 5 has 5, 4 entries, whereas the bucket size is 2. So, it has a, it has to take care of overflow bucket to go through. So, additional block access, and if you increase this more, it might increase even more.

(Refer Slide Time: 15:00)

Deficiencies of Static Hashing

- In static hashing, function h maps search-key values to a fixed set of addresses. Databases grow or shrink with time
 - If initial number of buckets is too small, and file grows, performance degrades due to too many overflows
 - If space is allocated for anticipated growth, a significant amount is wasted initially (and buckets will be underfull).
 - If database shrinks, again space will be wasted
- One solution: periodic re-organization of the file with a new hash function
 - Expensive, disrupts normal operations
- Better solution: allow the number of buckets to be modified dynamically

h: D

So, in summary, the static hashing which maps search key values to a fixed set of bucket addresses becomes a problem because once a bucket starts getting more and more records concentrate on that, you have to make that overflow chain, which is so, and also the fact that the database is a dynamic entity.

So, initially a database is small, so you need a small number of bucket, as it grows, the performance will degrade. If you allocate initially itself that I will allocate these many buckets with the anticipated growth, then there is a significant amount of space that is wasted initially, and in fact, in many cases you do not know what that growth could be. So, the basic problem lies with having to deal with a fixed set of buckets in case of the static hashing.

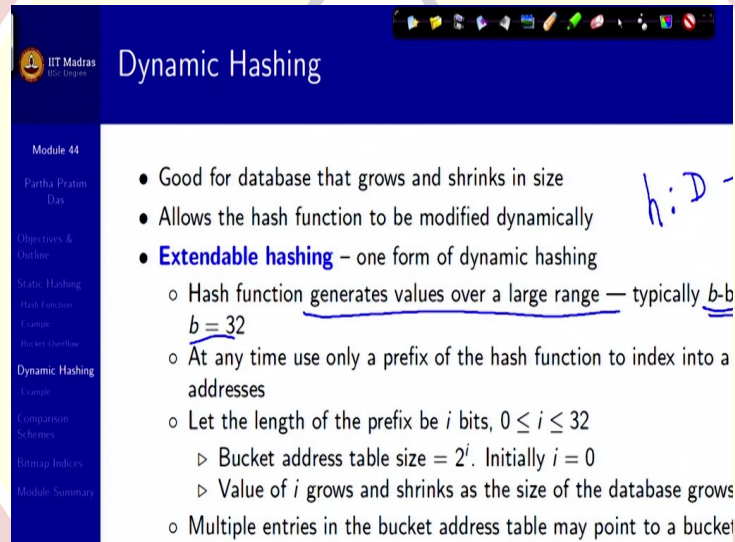
So, what would be preferable if we could change the number of buckets dynamically? Now, the number of buckets just to recall is the range of the hash function. So, if I change this n naturally, I will have to change the hashing function and rehash everything. For example, in the last ID case, we were doing adding the digits and doing a modulo 8, to increase we at one point we decide to do modulo say 12 that will increase 4 buckets, 50 percent. But as we do that, the hash values will change. So, all reorganization will be required.

So, the static hashing has a problem with the growth it is good fast for I mean kind of if your database data is more or less static, which is normally would not be the case for most databases, but in general it will have a problem with this growth.

(Refer Slide Time: 17:19)

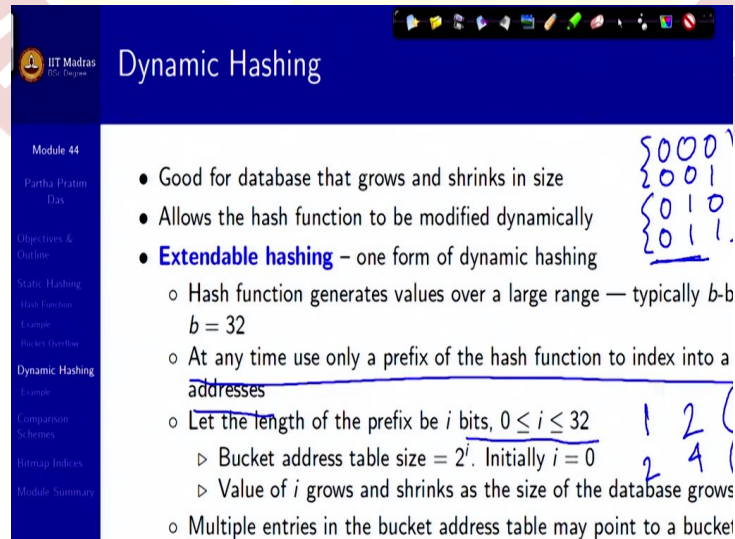


Slide 1: Dynamic Hashing. The slide features a blue header with the IIT Madras logo and the title "Dynamic Hashing". A sidebar on the left lists the module contents, with "Dynamic Hashing" highlighted. The main content area is mostly blank, with the title "Dynamic Hashing" written in red text in the center.



Slide 2: Dynamic Hashing. The slide features a blue header with the IIT Madras logo and the title "Dynamic Hashing". A sidebar on the left lists the module contents, with "Dynamic Hashing" highlighted. The main content area contains a list of bullet points. Handwritten notes in blue ink are present: "h:D" next to the first bullet point, and "b=32" underlined next to the second bullet point.

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** – one form of dynamic hashing
 - Hash function generates values over a large range — typically $b=32$
 - At any time use only a prefix of the hash function to index into addresses
 - Let the length of the prefix be i bits, $0 \leq i \leq 32$
 - ▷ Bucket address table size = 2^i . Initially $i = 0$
 - ▷ Value of i grows and shrinks as the size of the database grows
 - Multiple entries in the bucket address table may point to a bucket



Slide 3: Dynamic Hashing. The slide features a blue header with the IIT Madras logo and the title "Dynamic Hashing". A sidebar on the left lists the module contents, with "Dynamic Hashing" highlighted. The main content area contains a list of bullet points. Handwritten notes in blue ink are present: "50001", "0001", "010", "011" next to the first bullet point, and "1 2", "2 4" next to the third bullet point.

- Good for database that grows and shrinks in size
- Allows the hash function to be modified dynamically
- **Extendable hashing** – one form of dynamic hashing
 - Hash function generates values over a large range — typically $b=32$
 - At any time use only a prefix of the hash function to index into addresses
 - Let the length of the prefix be i bits, $0 \leq i \leq 32$
 - ▷ Bucket address table size = 2^i . Initially $i = 0$
 - ▷ Value of i grows and shrinks as the size of the database grows
 - Multiple entries in the bucket address table may point to a bucket

So, we move on to dynamic hashing or also called extendable hashing. In extendable hashing hash function is not taught to be changed, you cannot do that. It generates a value over a large range that is as if you are thinking that there is a virtual space of buckets. So, you can say there is a b -bit integer, the range is a b -bit integer. So, what we are saying is hash function goes from D to 2 to the power b , if it is a b -bit integer, then there are 2 to the power b possibilities and say, with the value 32 .

So, which is, it is quite unlikely that most databases will require so many buckets will not. But this is what is you start with so that that is your function. That is how you design the function. The trick lies in the next step. You have 32 bits, but you do not use all of them. You just use a few prefix bits of the index. Rest of them you ignore. So, maybe if you are, if you want to have 4 buckets, then you use 2 bits from the front and ignore the remaining 30 . What does that mean? That means that every 2 to the power 30 buckets are getting collapsed into 1 bucket. They are getting collapse into 1 bucket.

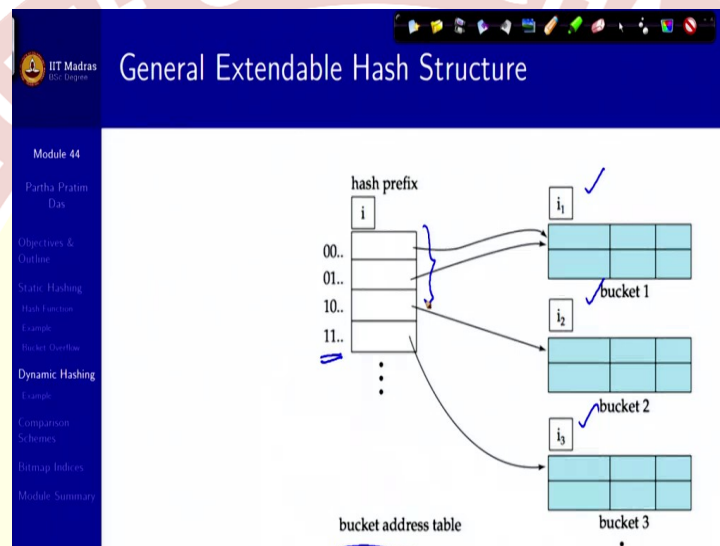
So, just if you would be comfortable with an example, let us say my B is 3 , my B is 3 . If I use just a single bit prefix, then I have a bucket 0 and a bucket 1 ; 2 buckets. So, 0123 all these 4 buckets virtual buckets mapped to buckets 0 . 4567 , all these virtual buckets map to bucket 1 . Instead, if I use 2 bits, instead of using 1 , then what happens, I have bucket 0 here with 0 and 1 , I have bucket 1 here, with 1 and 2 , I have bucket 2 here with 4 and 5 , I have bucket 3 here with 6 and 7 .

So, you can see by, I mean, what I am trying to show you is by the choice of how many bits in the prefix you take, you do not need to recompute the hash function by but by changing this number of bits you take from the prefix can actually change the buckets that you use. Now, so, it could be anywhere between 0 to 1 bit to 32 bits, and as i grows, the number of prefix bits that you are using as that grows, then you will shrink the database size. I mean, I am sorry, as that grows, you have more number of buckets available. And if that shrinks, there are less number of buckets available.

So, the database size based on the growth and shrinkage of the database size, you could control this value of i . Now naturally, even in this it is possible that so, you have now you have these set of prefixes that you have chosen, and then you take the buckets mapped to them. Now, it is possible that all the prefixes that you have taken may not have separate bucket for the simple reason that when you make your choice of the prefix bits, you can use 1 , you can use 2 , you can use 3 like that.

But for that, the number of buckets you need is 2 4 8 like that, as you increase 1 bit in the prefix, your number of bucket requirement doubles. But you may not actually have that much of data to double or that much of space to allocate for buckets. So, you may be using 3 bits requiring 8 buckets, but you may actually be using 5 buckets. So, the actual number of buckets may be less than 2 to the power i. So, what does that mean that some of these values from this i bits will map have to map to the same bucket. That is the point we are talking of here.

(Refer Slide Time: 23:10)



For example, here is an example with you have 2 bits used. So, the bucket address table this is called the bucket address table where you keep the hash value and the corresponding bucket pointer. So, in this bucket address table, you have 4 pointers, but you have only 3 buckets. So, these two and we will see how that happens. These two are mapped to the same bucket whereas 2 and 3 are mapped to two different buckets.

(Refer Slide Time: 23:51)

Use of Extendable Hash Structure

- Each bucket j stores a value i_j
 - All the entries that point to the same bucket have the same values
- To locate the bucket containing search-key K_j
 - Compute $h(K_j) = X$
 - Use the first i high order bits of X as a displacement into bucket j follow the pointer to appropriate bucket
- To insert a record with search-key value K_j
 - Follow same procedure as look-up and locate the bucket, say j
 - If there is room in the bucket j insert record in the bucket
 - Else the bucket must be split and insertion re-attempted (next slide)
 - ▷ Overflow buckets used instead in some cases (will see shortly)

Insertion in Extendable Hash Structure

To split a bucket j when inserting record with search-key K_j

- If $i > i_j$ (more than one pointer to bucket j)
 - Allocate a new bucket z , and set $i_j = i_z = (i_j + 1)$
 - Update the second half of the bucket address table entries originally pointing to j to point to z
 - Remove each record in bucket j and reinsert (in j or z)
 - Recompute new bucket for K_j and insert record in the bucket (if required if the bucket is still full)
- If $i = i_j$ (only one pointer to bucket j)
 - If i reaches some limit b , or too many splits have happened in this bucket, use an overflow bucket
 - Else

So, let us see I mean instead of going through the extended theory, let us quickly get to a point where we can, now searching is quite obvious, you get to this get to the index, compute the hash, take the prefix as many bits as we are using now, go to that bucket address table, go to the bucket and you have everything.

Now, when you are trying to insert a record that means you might need more bucket. There are two possibilities, one is the number of prefix you are using is greater than what you actually have in terms of the buckets. So, it is possible that you can allocate a new bucket as you will get a new bucket. You have to reorganize the records between the earlier bucket and the newly allocated bucket and recompute the new bucket.

If it is same that every index points to one bucket only, then you do not have buckets left. So, you may not be able to do a split for this insertion, you can create an overflow bucket. But the other option is you can increment the value of i you can use 1 more bit. So, the bucket address table will become doubled. And you will reorganize the buckets in the bucket address table so that your bucket address table is becoming double, but your value of actual buckets is increasing only by 1 because that is what you need.

So, you do not unnecessarily create empty buckets and push in there, you do not need to, rest of the that is $i+1 - i_j+1$ or $i-i_j$ number of buckets will of pointers will point to the same some of the same buckets as they exist.

(Refer Slide Time: 26:20)

Deletion in Extendable Hash Structure

- To delete a key value,
 - locate it in its bucket and remove it
 - The bucket itself can be removed if it becomes empty (with appropriate update to the bucket address table)
 - Coalescing of buckets can be done (can coalesce only with a "bucket" having same value of i_j and same $i_j - 1$ prefix, if it is present)
 - Decreasing bucket address table size is also possible
 - Note: decreasing bucket address table size is an expensive operation. It should be done only if number of buckets becomes much smaller than the bucket address table size.

Use of Extendable Hash Structure: Example

<i>dept_name</i>	<i>h(dept_name)</i>
Biology	0010 1101 1111 1011 0010 1100 0011 00
Comp. Sci.	1111 0001 0010 0100 1001 0011 0110 11
Elec. Eng.	0100 0011 1010 1100 1100 0110 1101 11
Finance	1010 0011 1010 0000 1100 0110 1001 11
History	1100 0111 1110 1101 1011 1111 0011 10
Music	0011 0101 1010 0110 1100 1001 1110 10
Physics	1001 1000 0011 1111 1001 1100 0000 00

Deletion happens by the same logic rather, I am keen to go to an example. So, let us start with this in the next couple of minutes. So, this is a 32 bit hash key generated for the department name. So, we will use this as our data.

(Refer Slide Time: 26:42)

Example (2)

Module 44
Partha Pratim Das
Objectives & Outline
Static Hashing
Hash Function
Example
Bucket Overflow
Dynamic Hashing
Comparison Schemes
Bitmap Indices
Module Summary

- Initial Hash structure; bucket size = 2

hash prefix

0

bucket address table

bucket 1

dept_name

Biology	0010 1101 11
Comp. Sci.	1111 0001 00
Elec. Eng.	0100 0011 10
Finance	1010 0011 10
History	1100 0111 11
Music	0011 0101 10
Physics	1001 1000 00

Insert "Mozart", "Srinivasan", and "Wu" records

76766	Crick
10101	Srinivasan
45565	Katz
83821	Brandt

So, this is the initial you have 0, this is the initial bucket, this is the data to insert. So, I will insert Mozart, Srinivasan and Wu in this order. So, Mozart is music, Srinivasan is computer science and Wu is in finance. Finance, music, computer science 1 bit, 1 bit, 1 bit. So, you can see that finance and computer science will clash will be in the same bucket and music will be in a different bucket.

(Refer Slide Time: 27:20)

Example (3)

Module 44
Partha Pratim Das
Objectives & Outline
Static Hashing
Hash Function
Example
Bucket Overflow
Dynamic Hashing
Comparison Schemes
Bitmap Indices
Module Summary

- Hash structure after insertion of "Mozart", "Srinivasan", and "Wu" records

hash prefix

1

bucket address table

1

15151 Mozart Music 40000

10101 Srinivasan Comp. Sci. 90000

12121 Wu Finance 90000

dept_name

Biology	0010 1101 11
Comp. Sci.	1111 0001 00
Elec. Eng.	0100 0011 10
Finance	1010 0011 10
History	1100 0111 11
Music	0011 0101 10
Physics	1001 1000 00

Insert Einstein record

76766	Crick
10101	Srinivasan
45565	Katz
83821	Brandt

So, that is what has happened they are and this is in music is in a different bucket because it has a value 0 and computer science and finance has a value 1. This 1 marks how many bits are being used to come to this bucket. So, in both of these, this is 1, now we want to insert Einstein onto this.

(Refer Slide Time: 27:49)

Example (4)

- Hash structure after insertion of "Einstein" record

dept_name

Biology	0010 1101 11
Comp. Sci.	1111 0001 00
Elec. Eng.	1010 0011 10
Finance	1100 0011 10
History	1100 0111 11
Music	0011 0101 10
Physics	1001 1000 00

76766 Crick
10101 Srinivasan
45565 Katz
83821 Brandt

Example (3)

- Hash structure after insertion of "Mozart", "Srinivasan", and "Wu" records

dept_name

Biology	0010 1101 11
Comp. Sci.	1111 0001 00
Elec. Eng.	1010 0011 10
Finance	1100 0011 10
History	1100 0111 11
Music	0011 0101 10
Physics	1001 1000 00

76766 Crick
10101 Srinivasan
45565 Katz
83821 Brandt

So, what is Einstein? Who is, I mean where is Einstein, is in physics, physics, this is 1. So, when it comes to that, if we go back a slide. If we go back a slide, we will see that physics is 1, computer science is 1, finance is also 1. So, Einstein needs to go in here, where we have assumed that every bucket has 2, size of 2. So, this bucket is already full. So, Einstein cannot go in here.

So, what I do is I increase the value of i that is the number of prefix bits we are using. And so therefore, your bucket address table now has 4 entries. You have, you had 2 buckets and because of this need to insert a new record you need another bucket so you have created another bucket. Now let us look at the values.

Physics is 1 0, that is 2, music is 0 0 that is 0. Finance is 1 0, that is 2, computer science is 1 1 that is 3. So, this computer science comes here, this is 3 2 1 0, physics and finance come here music remains in 0, no change. And so, this we had to create because of inserting this record so this goes separately with index 2. This also is having 2 but we do not need to create a separate bucket for 1. The value 1 that is which will mean a prefix of 0 1 that is till we get electrical engineering actually. And we let both of these map to the same bucket.

So, if you look in carefully, here we are saying 1 that is 1 bit is used because the second bit is ignored, because you have multiple are mapping here, so, 0 and 1 out of 0 and 1 you are just using the 0, from 0 0 and 0 1, you are representing by 0, so, it is 1 bit whereas, these two are 2 bits, it is as simple as that. So, you can insert Gold and El Said records.

(Refer Slide Time: 30:43)

Example (5)

- Hash structure after insertion of "Gold" and "El Said" records

hash prefix	bucket	dept_name	emp_name	emp_id
0	15151	Mozart	Music	40000
1	22222	Einstein	Physics	95000
2	33456	Gold	Physics	87000
3	12121	Wu	Finance	90000
4	10101	Srinivasan	Comp. Sci.	65000
5	32343	El Said	History	60000

dept_name	emp_name	emp_id
Biology	0010 1101 11	
Comp. Sci.	1111 0001 00	
Elec. Eng.	0100 0011 10	
Finance	1010 0011 10	
History	1100 0111 11	
Music	0011 0101 10	
Physics	1001 1000 00	

emp_id	emp_name	dept_name
76766	Crick	
10101	Srinivasan	
43565	Katz	
83821	Brandt	

Module 44

Partha Pratim Das

Objectives & Outline

Static Hashing

Hash Function

Example

Bucket Overflow

Dynamic Hashing

Comparison Schemes

Bitmap Indexes

Module Summary

Example (6)

- Hash structure after insertion of "Katz" record

Gold is in physics clearly goes in here. El Said is in history, is in history, History is 3. So, that that can go in here. And you can see that again, we increase the number of bits again, this is using 1 prefix only. So, 4 values are here, this is using all the 3. So, 1. This is using all the 3, this is using just 2. So, you have 2 values coming here. So, it keeps on going in this manner, you can step by step work out, we are including all of these.

(Refer Slide Time: 31:41)

Module 44

Partha Pratim Das

Objectives & Outline

Static Hashing

Hash Function

Example

Bucket Overflow

Dynamic Hashing

Comparison Schemes

Bitmap Indexes

Module Summary

Example (7)

- Hash structure after insertion of "Singh", "Califrieri", "Crick", "Brandt" records

IIT Madras
For Engineers

Example (8)

- Hash structure after insertion of "Kim" record

dept_name	
Biology	0010 1101 11
Comp. Sci.	1111 0001 00
Elec. Eng.	0100 0011 10
Finance	1010 0011 10
History	1100 0111 11
Music	0011 0101 10
Physics	1001 1000 00

76766	Crick
10101	Srinivasan
45565	Kate
83821	Brandt
56789	Califeri

Now, when you included all of these, you again had a overflow with brand, because computer science needs another block. Now, you may use 4 bits, but you chose not to immediately increase to 4 bits. Because you can see that well. There is already a number of, unused stuff here, but it is just that the keys are biased towards this side. So, instead of increasing the bucket address table size, reorganizing it, you use a overflow bucket. So, it is kind of a trade-off between the two as to which one you want to, you want to do at different stages. Finally, you insert Kim.

(Refer Slide Time: 32:43)

IIT Madras
For Engineers

Comparison Schemes

Module 44
Partha Pratim Das
Objectives & Outline
Static Hashing
Hash Function
Example
Bucket Overflow
Dynamic Hashing
Example
Comparison Schemes
Bitmap Indexes
Module Summary

Comparison Schemes



Extendable Hashing vs. Other Schemes

Module 44
Partha Pratim Das
Objectives & Outline
Static Hashing
Hash Function
Example
Bucket Overflow
Dynamic Hashing
Example
Comparison Schemes
Bitmap Indices
Module Summary

- Benefits of extendable hashing:
 - Hash performance does not degrade with growth of file
 - Minimal space overhead
- Disadvantages of extendable hashing
 - Extra level of indirection to find desired record
 - Bucket address table may itself become very big (larger than men)
 - ▷ Cannot allocate very large contiguous areas on disk either
 - ▷ Solution: B⁺-tree structure to locate desired record in bucket
 - Changing size of bucket address table is an expensive operation
- Linear hashing is an alternative mechanism
 - Allows incremental growth of its directory (equivalent to bucket a

And go through the steps carefully so that you will know that what will. So, the comparison is pretty straightforward. Obviously, the dynamic scheme works better. Though change of the bucket address table is an expensive operation and has to be done with judgment.

(Refer Slide Time: 33:06)



Comparison of Ordered Indexing and Hashing

Module 44
Partha Pratim Das
Objectives & Outline
Static Hashing
Hash Function
Example
Bucket Overflow
Dynamic Hashing
Example
Comparison Schemes
Bitmap Indices
Module Summary

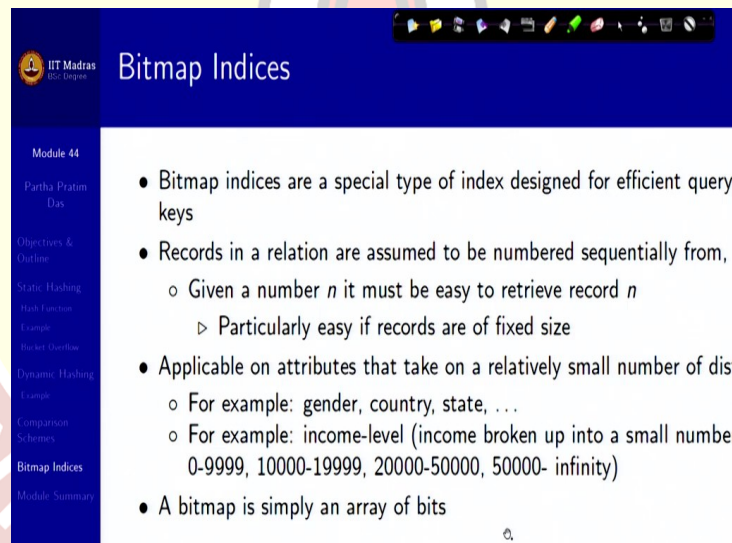
- Cost of periodic re-organization
- Relative frequency of insertions and deletions
- Is it desirable to optimize average access time at the expense of worst
- Expected type of queries:
 - Hashing is generally better at retrieving records having a specified
 - If range queries are common, ordered indices are to be preferred
- In practice:
 - PostgreSQL supports hash indices, but discourages use due to po
 - Oracle supports static hash organization, but not hash indices
 - SQLServer supports only B⁺-trees

And comparison of ordered indexing and hashing clearly shows that hashing would that way be quite good because it does not require the cost of periodic reorganization, relative frequency of insertions and deletions can be considered for that and depends on also the kind of queries you have, like hashing is good to retrieve records with a specific value because you need to hash.

Range queries are will be good with ordered indexing, because you need to go over a certain range. In spite of the power of the hashing in practice, though, it is not so widely used in the

big databases like I have just given some information of what so these are the schemes exist, but this is really what most of your major database provider actually do.

(Refer Slide Time: 34:14)





IIT Madras
For Engineers

Bitmap Indices (2)

Module 44

Partha Pratim Das

Objectives & Outline

State Hashing

Hash Functions

Example

Bitmap Overview

Dynamic Hashing

Example

Comparison Schemes

Bitmap Indices

Module Summary

- In its simplest form a bitmap index on an attribute has a bitmap for each attribute
 - Bitmap has as many bits as records
 - In a bitmap for value v, the bit for a record is 1 if the record has value v for attribute, and is 0 otherwise

record number	ID	gender	income_level
0	76766	m	L1
1	22222	f	L2
2	12121	f	L1

Bitmaps for gender

m

f

10010
01101

Bitmaps for income_level

L1

L2

L3

10100
01000
00000

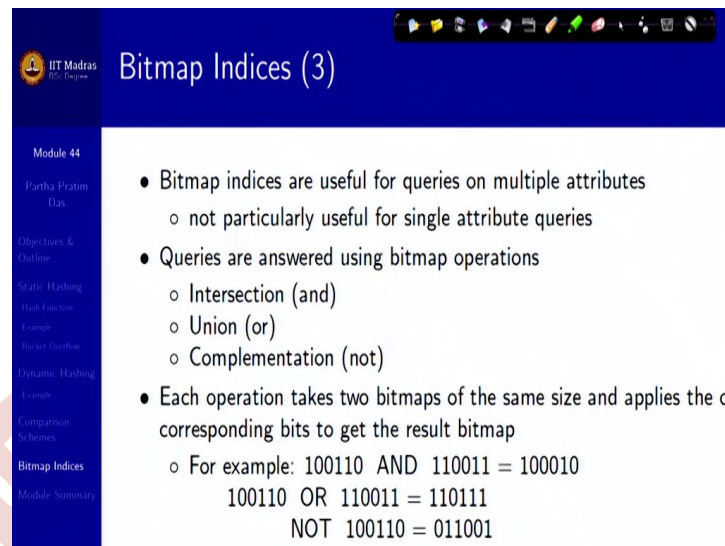
One small closing concept this is called a bitmap index. The idea of a bitmap index is certain attributes have a fixed set of, fixed and small set of values like gender, only two possible values or maybe few more, The country's 170-200 value, states of a country maybe 100 values, certain income levels, maybe 20 income levels and so on.

So, why, since when you have this small set of enumerated data like this for an attribute, then the idea is why do you keep on writing that male, male, male, male, female, female, female, female, female, like that? Why do you replicate? Instead, you could keep this in a different way, you can just turn the table instead of mentioning the value of an attribute for a record, instead you look at from the attribute's perspective, you look at whether a given attribute value is present for that record. So, simple example.

So, number the records in some arbitrary sequential order, and then you try to see, this is going from left to right, I am looking at so, I have a bitmap, which is the number of records that I have and if m male is present, I put a 1, this is female, I put a 0, this is female, I put a 0 this is male, I put a 1. Similarly, in female, I will just have the other places as well.

So, when you have to search for a particular, I mean is male, all that you need to do is to collect the bits, which are the which are indicative whose positions are indicative of the record number. Similar thing you can do for income level.

(Refer Slide Time: 36:23)

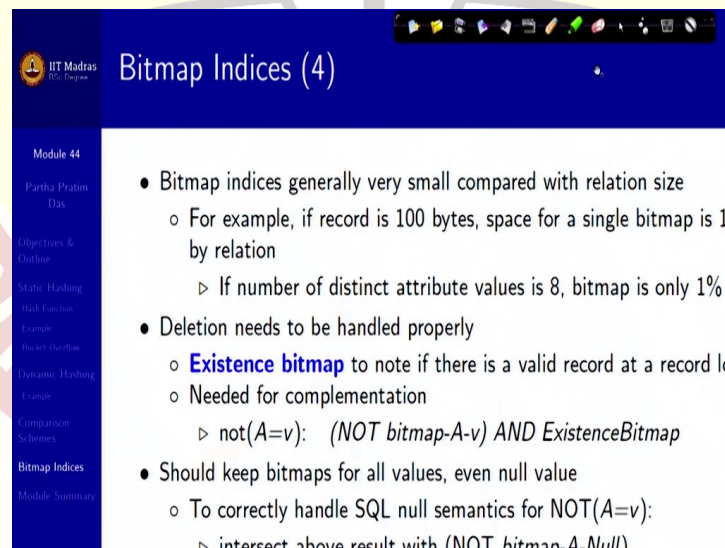


Bitmap Indices (3)

- Bitmap indices are useful for queries on multiple attributes
 - not particularly useful for single attribute queries
- Queries are answered using bitmap operations
 - Intersection (and)
 - Union (or)
 - Complementation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap
 - For example: $100110 \text{ AND } 110011 = 100010$
 $100110 \text{ OR } 110011 = 110111$
 $\text{NOT } 100110 = 011001$

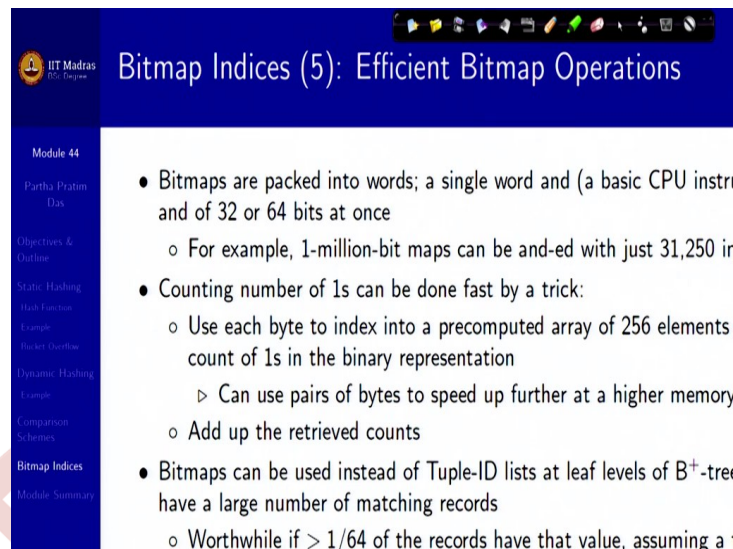
What is actually far more interesting is you can actually do and if you want to say the male and income level 14 A, then the two bitmaps of the same size the number of records in the collection, you can just AND them. So, logically AND them and wherever you get a 1 is the other position other indices of the records which have that.

(Refer Slide Time: 36:47)



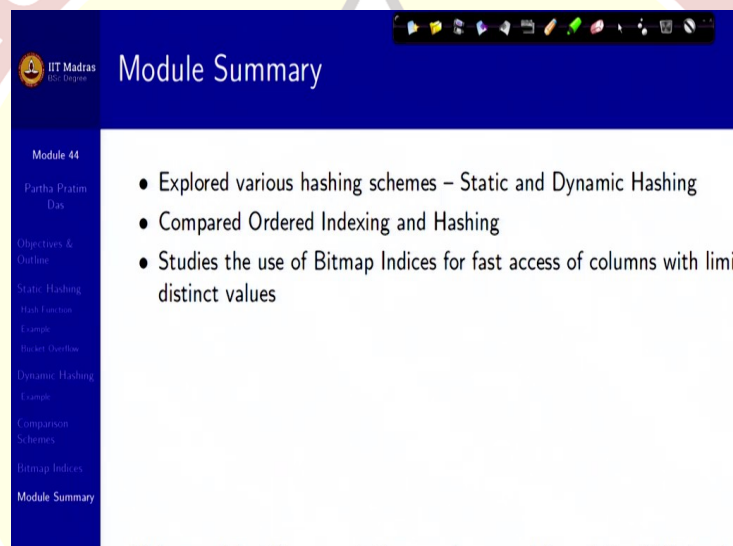
Bitmap Indices (4)

- Bitmap indices generally very small compared with relation size
 - For example, if record is 100 bytes, space for a single bitmap is 1/100 by relation
 - ▷ If number of distinct attribute values is 8, bitmap is only 1% of relation
- Deletion needs to be handled properly
 - **Existence bitmap** to note if there is a valid record at a record location
 - Needed for complementation
 - ▷ $\text{not}(A=v): (\text{NOT bitmap-}A-v) \text{ AND ExistenceBitmap}$
- Should keep bitmaps for all values, even null value
 - To correctly handle SQL null semantics for $\text{NOT}(A=v)$:
 - ▷ intersect above result with $(\text{NOT bitmap-}A-\text{Null})$



Bitmap Indices (5): Efficient Bitmap Operations

- Bitmaps are packed into words; a single word and (a basic CPU instruction) and of 32 or 64 bits at once
 - For example, 1-million-bit maps can be and-ed with just 31,250 instructions
- Counting number of 1s can be done fast by a trick:
 - Use each byte to index into a precomputed array of 256 elements containing count of 1s in the binary representation
 - ▷ Can use pairs of bytes to speed up further at a higher memory cost
 - Add up the retrieved counts
- Bitmaps can be used instead of Tuple-ID lists at leaf levels of B⁺-trees
 - have a large number of matching records
 - Worthwhile if $> 1/64$ of the records have that value, assuming a 64-bit word



Module Summary

- Explored various hashing schemes – Static and Dynamic Hashing
- Compared Ordered Indexing and Hashing
- Studies the use of Bitmap Indices for fast access of columns with limited distinct values

So, these are you know, more details in discussion. And there are several algorithms to efficiently compute this through lookup tables and so on. So, bitmap is a nice idea to expedite the access of a number of attribute, possible attributes which have fixed small set of possible values. So, with that, we come to the end of this module. Thank you very much for your attention and we meet in the next module.